

AAH Harness Capabilities for the GNFR Spend Visibility Build

What the harness integrates with, what it enforces, and what it would take to wire it to our stack.

AUDIENCE	Frontend, backend, AI/ML, data and security engineers on the GNFR build
COMPANION TO	Building GNFR Spend Visibility on the Ascend Agentic Harness (business case)
BASIS	AAH source tree; the agentic architecture deep-dive; the spend-visibility-mvp build
SCOPE	Mechanism and integration surface only — no commercial or delivery-timeline argument

§ 01

Model Context Protocol — two distinct roles

AAH touches MCP in two places that are easy to conflate. Both matter for this build, and they are wired differently.

1.1 • The harness consuming MCP servers

At session start, `core.mcp_servers.mcp_server_loader` shells out to `claude mcp list`, parses the health output, and keeps only servers reporting `Connected`. Each known server has a reference document under `mcp_servers/references/` whose frontmatter declares the phases it applies to, an `action_hint`, and its tool list. `load_impl_context` folds that summary into the implementer's startup context — so the agent begins a feature already knowing which tool servers are live and when to reach for them, rather than rediscovering them per session.

Registration is declarative. `aah/hooks/mcp-servers.json` is the source of truth that `aah install / aah setup` merges into the host MCP config. The merge is **additive and idempotent**, keyed by an `_aah` marker: a server is added only if its name is absent, so a server the team defined itself is never overwritten, and `uninstall` removes exactly what AAH recorded. Adding a server is one JSON entry — no Python change.

What ships today, and what this means for Databricks

The default config registers exactly one server: Playwright. There is no Databricks, Azure or Content Safety entry out of the box. Wiring our data plane in is a configuration addition rather than a code change — an entry in `mcp-servers.json` plus a reference doc declaring its tools and phases makes the Databricks MCP tool surface visible to the feature implementer at build time. Note one operational constraint from the source: Claude Code connects MCP servers only at session **startup**, so `aah setup` has to run before the session boots for same-session availability.

1.2 • The harness building MCP into our agent

The mcp-integration skill is the other direction — connecting the agent we are building to MCP tool servers. It runs in two modes, and the distinction is important for unattended builds. In **AAH mode** the feature implementer runs it non-interactively: it must not call `AskUserQuestion` or block waiting to show output. Its inputs come from artefacts instead — whether MCP is in scope at all comes from the discuss registry's tool-protocol decision, which server and URL come from the feature YAML's integration notes, and the agent framework comes from the recorded framework-selection decision. Completion is defined as passing functional tests against the feature's acceptance criteria, explicitly **not** "show the tool listing to the user".

For our build this maps cleanly onto the three tools the deep-dive defines — `query_spend_table`, `find_supplier` and `query_semantic_planner_rag` — all of which are MCP calls against the Databricks Managed MCP endpoint over Private Link, with typed schemas and parameterised SQL. That is Path A of the skill (connect to an existing server with auth), not Path C (create one).

§ 02

Issue trackers and documentation systems

This is where the honest answer differs from the hopeful one, so it is worth stating plainly before the capability description.

Confluence is not supported, and Jira is a stub

There is no Confluence adapter anywhere in the harness, and no extension point for one in the version-control subsystem — that subsystem models **work items**, not pages. Jira exists only as a placeholder package whose `__init__.py` documents the contract but ships no implementation. **GitHub is the only working provider**, with a real `client.py`, `mapper.py` and `provider.py`. Adding Jira is a bounded piece of work against a defined interface: implement those three files as a `TrackerProvider` subclass, call `register("jira", JiraProvider)`, and import the package — the source states explicitly that no core changes are required. Reaching Confluence at all would mean treating it as an MCP context source under §1.1, not as a tracker.

2.1 • What the sync engine actually does

Where a provider does exist, the mechanism is stronger than the usual one-way export. `core.version_control.engine` performs a **three-way reconciliation** per feature, comparing three snapshots: `base` (the last-synced values held in a ledger), `local` (the current feature file, treated as source of truth) and `remote` (the current tracker item, treated as a human proposal).

For each synced field it computes whether local changed and whether remote changed against base. That two-by-two produces a verdict — `NO-OP`, `OUTBOUND`, `INBOUND` or `CONFLICT` — which aggregates to a feature-level delta type, with genuine conflicts written to a conflict store rather than silently resolved. The engine mutates nothing; application is a separate step.

The practical effect: a sourcing product owner editing an issue description and an engineer editing the feature spec do not clobber each other, and the divergence surfaces as a named conflict. The build skill also acts on this — on resume it runs a full resync, and where a tracker issue triggered a run it comments the decision back and routes the fix through aah - fix, closing the issue only when fully resolved.

§ 03

Enforcement at the tool call, not at review

The guards are not lint run after the fact. They are registered as PreToolUse hooks, so they intercept the agent's Bash, Write, Edit and Read calls before the operation executes. A blocked guard aborts the tool call outright.

GUARD	FIRES ON	WHAT IT DOES
secrets_guard / secrets_commit_guard	Read, Write, Edit, Bash	Intercepts secret material reaching disk or a commit.
package_existence_guard	Bash	Blocks batch installs whose declared names do not resolve in the committed lockfile. Deterministic and offline by design — no registry lookup, deliberately avoiding the SSRF surface a live check would reintroduce. This is the defence against a hallucinated or slopsquatted dependency name.
dependency_policy_guard / dependency_license_guard	Bash	Version policy and licence policy on dependency changes — the sibling checks to existence.
no_mocks_guard	Bash	Detects mock-framework installation. Runs in relaxed mode : it warns and allows. The hard prohibition lives in the project ruleset and in the functional-test requirement at the QA gate, not in this hook.
verification_artifact_guard	Write, Edit, MultiEdit	Blocks any write to a test-result, runtime-result, quality-result, checkpoint or validation path that did not come through the official attested writer.
validate_feature_list	Write	Structural edits to feature-list.json are refused; only status changes pass.
execution_risk_monitor	Post-tool	Watches executed commands for risk signals after the fact.

3.1 · Attestation — why "the tests passed" is verifiable

The threat model is stated explicitly in the source, and it is the right one for a system built by AI agents: an agent can write {"passed": true} straight into a result file, and the orchestrator advances because nothing re-runs the work.

The defence is two-sided. At the **write boundary**, every legitimate writer — the QA report, the regression suite, the per-feature test runner, checkpoint validation, quality checks, runtime results — routes through `attestation.write_attested`, which sets an environment marker for the duration of the write; `verification_artifact_guard` blocks anything landing on those paths without it. At the **read boundary**, each artefact carries an HMAC signature keyed by a project-local secret at `.aah/build/.attestation-secret` — created once, mode 0600, gitignored, and resolved through Git's common directory so every worktree shares one key. The orchestrator verifies signatures when reading.

Either half is sufficient alone. Together, forging a green gate requires defeating cryptographic signing **and** a perimeter hook simultaneously. The same pattern extends to the codemap refresh, whose result is accepted only if the attestation verifies, the recorded HEAD sha still matches the branch, and the recorded database hash still matches the file on disk — freshness is re-derived rather than trusted from a marker.

Direct relevance to the governance review

We are submitting an AI system, built substantially by AI agents, to an AI governance board. The distinction between *an agent asserting that tests passed* and *a signed artefact produced by a deterministic runner* is exactly the distinction that review will probe. This mechanism is the difference between the two.

§ 04

Context engineering — how the implementer knows the codebase

A recurring failure mode with AI-assisted delivery is that each session starts cold and re-derives (or re-invents) the codebase's conventions. AAH addresses this with a deterministic startup chain and three accumulating artefacts.

- **SessionStart hook chain.** `aah setup` refreshes skills and agents, `run_init_check` health-checks the environment, `load_impl_context` assembles project context, orphaned worktrees are cleaned, git merge drivers are registered, and the attestation key is ensured. This runs before the model sees anything — context loading is a script, not a reminder.
- **Knowledge graph.** The codebase profiler's inventory is enriched by an AST extractor into a NetworkX directed graph with typed edges — `calls`, `imports`, `inherits`, `implements`, `uses`. It uses tree-sitter for Python, TypeScript, Go and Java where the grammars are installed, and falls back to regex extraction where they are not. It parses only files the profiler flagged as key — entry points, APIs, models, hotspots — capped by default at 100, so the graph stays a map rather than a dump.
- **Codemap.** A symbol and relation index refreshed per wave as an isolated subprocess, so an indexer crash cannot take the orchestrator with it, with the attested freshness check described above.
- **Two-tier expertise.** Tier 1 (`expertise.yaml`) holds project-wide orientation, a style guide and known concerns; Tier 2 holds per-module deep knowledge under `domains/`. What makes it usable is the hard budget: quick context capped at eight sentences, style guide at thirty lines, known concerns at fifteen entries, at most five technology patterns, five knowledge entries per wave and four integrations. Each known-concern entry must state when it triggers, what the symptom looks like and how to avoid it — actionable, not descriptive.

For a four-capability programme this is the mechanism that carries hard-won knowledge forward. M1 through M4 share one ontology — spend facts, contracts, contract coverage, compliance exceptions. The conventions and traps discovered while building Spend Visibility become startup context for Invoice Leakage rather than folklore.

§ 05

Domain briefs and archetype playbooks

5.1 • Domain briefs

Domain Briefs are a hierarchical taxonomy of functional knowledge — industry, sub-industry, process area, process step — where a leaf inherits its ancestors' content through a merge, and the resolved brief is injected into session context during the research and analysis phases. A brief carries processes, data entities, regulations, common patterns, glossary, typical integrations and KPIs.

What ships is financial-services only — commercial banking, retail banking, wealth management, accounting and finance. There is no retail or indirect-procurement branch. A GNFR brief would be new authoring under a new top-level node, carrying our sourcing vocabulary, the M1-M4 entity model and the compliance expectations. One-time, and it pays back across all four capabilities.

5.2 • The AI-applications playbook already describes our architecture

The MVP's manifest classifies it as the *ai-applications* archetype. That playbook's cross-cutting instruction reads as a summary of the deep-dive: build the skeleton deterministically and push stochastic behaviour into narrow, testable leaves; classify intents explicitly; ground factual answers with citations; use typed tool calls; gate high-stakes actions behind human confirmation; evaluate every prompt, model and retrieval change against a regression suite.

PLAYBOOK PATTERN	WHAT IT PRESCRIBES	WHERE OUR ARCHITECTURE ALREADY DOES IT
Intent-First Design	A typed intent taxonomy drives prompt selection and tool access; an explicit unknown-intent path returns a clarifying response. Anti-pattern: one system prompt handling every case.	The Orchestrator/Planner split — the Planner resolves intent to exactly one tool, and the Orchestrator's respond branch is the clarification path.
Grounded Generation Over Free Generation	Retrieve first, generate conditioned on retrieved context, require citations. Correctness control, not a nice-to-have.	Groundedness detection at <code>content_safety_out</code> , with the run's actual tool results passed as grounding sources and an ungrounded fallback message.
Structured Tool Calling	Typed tool calls with JSON-Schema-validated arguments; never parse actions out of narrative; never let the model compose raw requests.	The Planner returns strict JSON, the tool executor sanitises and re-validates before dispatch, and SQL is parameterised with bound placeholders rather than concatenated.

PLAYBOOK PATTERN	WHAT IT PRESCRIBES	WHERE OUR ARCHITECTURE ALREADY DOES IT
Tiered Response Quality	Route by complexity and stakes; fall back down the tiers on outage; keep routing in explicit config, not prompt logic.	The APIM backend pool — primary and secondary GPT-4.1 deployments with automatic failover, and Claude Sonnet available for complex semantic planning.
Deterministic Skeleton, Stochastic Leaves	Auth, routing, state transitions and validation stay deterministic; the model is a component, not the control plane.	The LangGraph node graph with deterministic conditional routing, Postgres checkpointing and the HitL interrupt gate.

The value here is not that the playbook teaches us something new. It is that the harness's archetype guidance and our architecture **already agree**, which turns the playbook into a conformance check the build phase applies automatically — with named anti-patterns that a reviewer can point at. Alongside it, the DDR playbook system and compliance-regime detection eliminate architectural options with recorded provenance based on the regimes the registry declares; our discuss registry currently carries SOC 2, GDPR, HIPAA and PCI-DSS pending reconciliation.

§ 06

Contract enforcement across three workloads

Our architecture keeps the Next.js frontend, the FastAPI service and the LangGraph agent as separate deployment units. The classic failure mode for that shape is silent drift between them. Three harness mechanisms attack it directly.

6.1 • API schema gate

`gates.validate_api_schema` runs four checks. It validates every module's schema file as OpenAPI 3.1. It enforces module-to-file consistency in both directions — every schema file must resolve to a module in the module map, and every API-exposing module must have a schema file. It requires every data-dependency line in the wireframes to resolve to a real `operationId`. And, opt-in via `--check-runtime`, it **boots the built application and diffs the live OpenAPI document against the declared schemas** — path and method existence, `operationId` match, and the 200-response schema reference name.

That last check is the one worth turning on for this build. It is callable from the standards gate, so code-versus-schema drift between the three workloads fails a merge instead of surfacing during UAT.

6.2 • Entity registry and fixture consistency

Feature YAMLS may declare an `entities` block. `plan.build_entity_registry` aggregates those into `.aah/entity-registry.json`, and its companion `mock_data_consistency_guard` blocks fixture writes that introduce entity names absent from the registry. For brownfield work it also harvests candidate names from existing fixtures so they are treated as canonical.

Applied to GNFR, this keeps the ontology honest across capabilities: if Spend Visibility calls it `spend_fact` and Invoice Leakage invents `spend fact` in a fixture, the write is refused rather than discovered three sprints later during parity validation.

6.3 • Generated smoke tests and the configuration contract

`plan.generate_smoke_tests` reads acceptance criteria, endpoints and scenarios from feature frontmatter and emits wave-level smoke definitions carrying a semantic request-and-assertions contract, so per-wave integration coverage is derived from the spec rather than hand-written. Separately, `plan.build_env_example` generates the committed environment catalogue that the boot-time checker validates against — the `ERR_CDR_78_EX_CONFIG` contract already visible in the MVP.

§ 07

Execution model

- **Dependency-ordered, one module at a time.** The plan phase produces a DAG and computes waves; the build skill reads the topological order and dispatches a feature implementer per module in sequence, doing red-to-green TDD plus a self code-review. The driver never writes code, never runs `raw pytest` or `npm test`, never runs `git merge` by hand, and never edits the feature list.
- **Runtime validation, not just unit green.** After each module the application is booted with its own start commands and probed. The probe utility redacts sensitive headers — authorization, cookie, token, key material — before anything is written to a result file. Where a UI exists, Playwright MCP drives it and captures a screenshot as evidence.
- **Branching and worktrees.** Feature work happens in isolated worktrees off develop; waves merge to integration/wave-N for cumulative regression, then promote to develop, with merge drivers registered at session start and orphaned worktrees swept automatically.
- **Everything routes through one door.** Any bug report, enhancement or reviewer comment goes through `aah-fix` rather than being acted on directly, which is what keeps the feature list, the progress file and the tracker in agreement.
- **Token accounting.** Per-project usage is aggregated to `.aah/audit/token-usage.json` from the raw counters, with a terminal report that breaks down share of project total — useful when the agent path's cost profile is itself under review.

§ 08

Gaps, and what closing them costs

GAP	CURRENT STATE	WHAT IT WOULD TAKE
Confluence	No adapter, and no seam for one — the version-control subsystem models work items, not pages.	Not a tracker-adapter problem. Reach it as an MCP context source instead: register a Confluence MCP server and a reference doc so requirement pages load as build context.

GAP	CURRENT STATE	WHAT IT WOULD TAKE
Jira	Documented placeholder package; GitHub is the only registered provider.	Implement <code>provider.py</code> , <code>mapper.py</code> and <code>client.py</code> against the <code>TrackerProvider</code> interface and register the factory. The source states no core changes are needed.
Azure / AKS deployment	Deploy routes cover AgentCore, Amplify, ECS, Cloud Run and a cloudless managed runtime. Azure appears only in cloud- and data-readiness probing.	Add an AKS route to the deploy routing config with a matching deploy engineer, or accept the standard infrastructure-as-code fallback and drive the release from our existing pipeline.
Databricks MCP registration	Only Playwright ships in the default MCP config.	One entry in <code>mcp-servers.json</code> plus a reference doc declaring tools and applicable phases. Configuration, not code.
Indirect-procurement domain brief	Financial-services taxonomy only.	Author a new top-level node and the GNFR leaf carrying our entity model, glossary, integrations and KPIs.
Mock prohibition strength	The hook warns rather than blocks; enforcement rests on the project ruleset and the functional-test requirement at QA.	Acceptable as-is if we hold the QA gate. Tighten the guard to blocking mode if we see mocks creeping into the suite.
Knowledge-graph fidelity	Without tree-sitter grammars installed, AST extraction degrades to regex and edge accuracy drops.	Install the grammars for Python and TypeScript in the build environment — a dependency addition.

§ 09

Recommended wiring sequence

Ordered by dependency, and by how much each unblocks the next.

01 Register the Databricks MCP server

Add it to `mcp-servers.json` with a reference doc naming `query_spend_table`, `find_supplier` and `query_semantic_planner_rag` and marking the build phase. This makes the tool surface visible to the implementer and is the precondition for the agent features being buildable unattended.

02 Install tree-sitter grammars in the build environment

Cheap, and it lifts the knowledge graph from regex approximation to real typed edges across the Python and TypeScript workloads.

03 Enable version control against the GitHub repository

Set the `version_control` block in the manifest to get three-way feature-to-issue sync, resume resync and decision comment-back. Scope the Jira adapter separately if the programme tracks work there.

04 **Declare the M1-M4 ontology entities in feature YAMLS**

Populates the entity registry so fixture drift across the four capabilities is blocked at write time rather than found during parity validation.

05 **Turn on runtime API-schema parity in the standards gate**

Three separately deployed workloads is precisely the drift case - - check - runtime exists for. Low effort, high catch rate.

06 **Author the GNFR domain brief**

New taxonomy node plus leaf. Feeds research and analysis context for every capability that follows.

07 **Decide the deployment route**

Add an AKS route to the routing config, or commit to the standard IaC fallback. Either is fine; leaving it undecided is not, because it is the one gap on the critical path.

The technical case reduces to one property. Every claim the build makes about itself — this test passed, this scan is clean, this schema matches, this package is real — is produced by a deterministic runner and verified at a boundary the agent does not control. For a system whose correctness argument has to survive a governance review, that property is worth more than any individual feature of the harness.